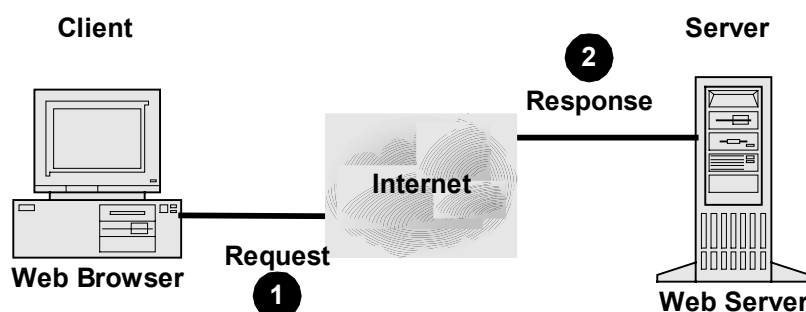


บทที่ 3

สถาปัตยกรรมบนฝั่งเซิร์ฟเวอร์

เอนเตอร์ไพรส์จาวาบีเอ็น (EJB) เป็นสถาปัตยกรรมคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์ (server-side component architecture) ซึ่งช่วยให้กระบวนการพัฒนาแอปพลิเคชันออบเจกต์แบบกระจาย (distributed object application) ในระดับเอนเตอร์ไพรส์ด้วยภาษาจาวานั้นทำได้ง่ายขึ้น ด้วย EJB เราสามารถสร้างแอปพลิเคชันที่สามารถเพิ่มขยายได้ (scalable) , มีความน่าเชื่อถือ (reliable) และมีความปลอดภัย (secure) โดยไม่ต้องเขียนโค้ดเกี่ยวกับการใช้งานออบเจกต์แบบกระจายซึ่งมีความซับซ้อนสูงด้วยตนเอง เราสามารถสร้างคอมโพเนนต์บนฝั่งเซิร์ฟเวอร์ด้วยภาษาจาวาได้อย่างง่ายดายและรวดเร็ว นอกจากนี้ EJB ยังถูกออกแบบขึ้นมาเพื่อสนับสนุนความสามารถในการเคลื่อนย้ายได้ (portability) และความสามารถนำกลับมาใช้ใหม่ได้ (reusability) ของ แอปพลิเคชันในแต่ละมิดเดิลแวร์ของแต่ละผู้ผลิต

แอปพลิเคชันบนฝั่งเซิร์ฟเวอร์ในยุคแรก ๆ นั้น ส่วนประกอบของเว็บยังไม่มี ความซับซ้อนเช่นในปัจจุบัน โดยใช้หลักการพื้นฐานของสถาปัตยกรรมแบบไคลเอนต์/เซิร์ฟเวอร์ ซึ่งเซิร์ฟเวอร์ในที่นี้ก็คือเว็บเซิร์ฟเวอร์ และไคลเอนต์ก็คือ เว็บเบราว์เซอร์ เช่น Netscape, Mosaic, Lynx , หรือ NCSA โดยเว็บเซิร์ฟเวอร์หนึ่ง ๆ จะให้บริการกับเว็บเบราว์เซอร์หลาย ๆ ตัวได้พร้อมกัน ดังรูปที่ 3-1



รูปที่ 3-1 การติดต่อระหว่างเว็บเบราว์เซอร์และเว็บเซิร์ฟเวอร์

แต่ข้อเสียของเว็บไซต์ในตอนนั้นก็คือ หน้าเว็บนั้นเป็น HTML ที่ตายตัว (static) ข้อมูลที่ปรากฏอยู่ไม่สามารถเปลี่ยนแปลงได้เลย เว็บเซิร์ฟเวอร์จะทำหน้าที่แค่เพียงส่งโค้ด HTML กลับมาเท่านั้น เทคโนโลยีที่เกิดขึ้นตามมาก็คือ CGI (Common Gateway Interface) ซึ่งใช้ในการสร้างหน้าเพจแบบไดนามิก เนื่องจากมันมีการประมวลผลก่อนที่จะส่งกลับไป

หลังจากที่ CGI ได้รับความนิยมจนกลายมาเป็นส่วนหนึ่งในมาตรฐานของเว็บแล้ว ก็ได้มีเทคโนโลยีที่ใกล้เคียงกันตามออกมามากมาย เช่น ASP , PHP , JSP , Servlet และอื่น ๆ อย่างไรก็ตามในโครงการนี้จะกล่าวถึงเฉพาะเทคโนโลยีแอปพลิเคชันทางฝั่งเซิร์ฟเวอร์ของจาวาเท่านั้น

ดีพลอยเมนต์บนฝั่งเซิร์ฟเวอร์ (server-side deployment) คือซอฟต์แวร์ที่ถูกเขียนขึ้นมาให้สามารถรองรับการใช้งานจากผู้ใช้ได้หลาย ๆ คนพร้อมกัน , มีความปลอดภัย , เชื่อถือได้ และมีประสิทธิภาพ ตัวอย่างของดีพลอยเมนต์บนฝั่งเซิร์ฟเวอร์ได้แก่

- ธนาคาร - เครื่องเอทีเอ็มจำนวนมากจะต้องติดต่อเข้าไปยังเครื่องเซิร์ฟเวอร์กลางของธนาคาร
- สาขาร้านค้า - เช่นร้านค้าในเครือข่ายของ Wal-Mart โดยร้านค้าแต่ละร้านจะส่งรายละเอียดการซื้อ สินค้าของลูกค้าแต่ละรายไปยังเซิร์ฟเวอร์กลางของ Wal-Mart
- ฝ่ายสนับสนุนลูกค้า - วิศวกรหลาย ๆ คนสามารถใช้เครื่องเทอร์มินอลเชื่อมต่อเข้ามาเซิร์ฟเวอร์กลางและนำข้อมูลเกี่ยวกับลูกค้าที่ถูกเก็บที่เซิร์ฟเวอร์ไปใช้งานได้
- เว็บไซต์ - ในเวลาใดเวลาหนึ่งจะมีผู้ใช้งานจำนวนมากทำการติดต่อเข้ามายังเว็บเซิร์ฟเวอร์ และเว็บเซิร์ฟเวอร์เหล่านี้จะต้องติดต่อไปยังเซิร์ฟเวอร์กลาง เพื่อดึงเอาข้อมูลและเรียกใช้การประมวลผลที่จำเป็น

การสร้างดีพลอยเมนต์บนฝั่งเซิร์ฟเวอร์ที่มีความสมบูรณ์นั้นไม่ได้เป็นเรื่องง่าย ปัญหาที่อาจเกิดขึ้นเช่น ปัญหาด้านความสามารถในการเพิ่มขยาย , การจัดการ , ความปลอดภัย , ความน่าเชื่อถือ และอื่น ๆ เนื่องจากว่า การทำงานของไคลเอนต์จำนวนมากนั้นจะขึ้นอยู่กับเซิร์ฟเวอร์เป็นสำคัญ การที่เซิร์ฟเวอร์ดาวน์โหลด , ทำงานช้าลง หรือมีผู้ประสงค์ร้ายลักลอบเข้าไปนั้น สามารถก่อให้เกิดความเสียหายได้อย่างมหาศาล ดังนั้นการสร้างดีพลอยเมนต์บนฝั่งเซิร์ฟเวอร์จะต้องค่อย ๆ ถูกสร้างขึ้นทีละชั้น และจะต้องมีการทดสอบอย่างรัดกุม

ในการทำดีพลอยเมนต์ที่ดีนั้น จะแบ่งซอฟต์แวร์ออกเป็นเลเยอร์ (layer) โดยในแต่ละเลเยอร์จะมีความรับผิดชอบในส่วนที่ดีพลอยเมนต์ที่แตกต่างกันออกไป และอาจประกอบด้วยหนึ่งคอมโพเนนต์หรือมากกว่าก็ได้ เลเยอร์เหล่านี้เป็นเพียงการแบ่งเลเยอร์ในทางความคิด (logical) เท่านั้น ซึ่งอาจแตกต่างจากการแบ่งหน้าที่ความรับผิดชอบของเครื่องคอมพิวเตอร์แต่ละเครื่องในทางกายภาพ (physical) ได้

ระบบที่แบ่งการทำงานเป็นเลเยอร์เป็นระบบที่ถูกออกแบบมาอย่างดี เนื่องจากแต่ละเลเยอร์นั้นจะมีหน้าที่รับผิดชอบที่แตกต่างกันออกไป การแบ่งเลเยอร์ทั่วไปจะแบ่งเลเยอร์เป็นดังนี้

พรีเซนต์เทชัน เลเยอร์ - ประกอบด้วยคอมโพเนนต์ที่จัดการกับการติดต่อกับผู้ใช้ (user interface) และการโต้ตอบกับผู้ใช้ (user interaction) พรีเซนเทชันเลเยอร์ของแอปพลิเคชันแบบ stand-alone อาจเขียนด้วยภาษาวิซวลเบสิก ส่วนพรีเซนเทชันเลเยอร์ของดีพลอยเมนต์แบบ Web-base นั้น จะใช้ Java Servlet , JSP , หรือ Java Applet

บิซิเนส ลอจิก เลเยอร์ - ประกอบด้วยคอมโพเนนต์ที่ใช้ในการแก้ปัญหาทางธุรกิจ คอมโพเนนต์เหล่านี้จะเป็น engine ที่มีความสามารถสูง เช่น engine ในการจัดการแคตตาล็อก หรือ คำนวณราคาสินค้า โดยปกติแล้วคอมโพเนนต์เหล่านี้จะต้องถูกเขียนขึ้นโดยภาษาที่มีคุณสมบัติ type-safe เช่น ภาษาจาวา หรือภาษา C++

ดาต้า เลเยอร์ - ถูกเรียกใช้โดยบิซิเนสลอจิกเลเยอร์เพื่อเก็บข้อมูลแบบถาวร โดยทั่วไปดาต้าเลเยอร์จะเป็นระบบฐานข้อมูล

ข้อดีของการแบ่งคอมโพเนนต์เป็นเลเยอร์คือ ทำให้สามารถเปลี่ยนแปลงแต่ละเลเยอร์ได้โดยจะไม่มีผลกระทบต่อเลเยอร์อื่น ๆ เช่น เราสามารถเปลี่ยนแปลงหน้าจออินเทอร์เน็ตเฟสในพรีเซนเทชันเลเยอร์ได้ โดยไม่มีผลกระทบต่อเลเยอร์อื่น

แต่การแบ่งลักษณะการทำงานออกเป็นเลเยอร์นี้ เป็นการแบ่งในทางความคิดเท่านั้น ส่วนในทางกายภาพ เราจะแบ่งออกเป็นหน่วยที่เรียกว่า “เทียร์” (tier) โดยสถาปัตยกรรมแบบหนึ่งเทียร์ คือการรวมเลเยอร์ทั้งสามเป็นส่วนเดียวทางกายภาพ , สถาปัตยกรรมแบบสองเทียร์ คือการแบ่งเลเยอร์ทั้งสามออกเป็นสองส่วนในทางกายภาพ และสถาปัตยกรรมแบบสามเทียร์ คือการแบ่งเลเยอร์ทั้งสามออกเป็นสามส่วนในทางกายภาพ

3.1 สถาปัตยกรรมแบบหนึ่งเทียร์ (1-Tier Architecture)

ในสถาปัตยกรรมนี้ ทั้งสามเลเยอร์จะรวมกันเป็นคอมโพเนนต์เดียวซึ่งโดยปกติจะเป็นโปรแกรมทำงานในเครื่อง เช่น เวิร์ดโปรเซสเซอร์ หรือสเปรดชีต ซึ่งหากมีการแก้ไขเปลี่ยนแปลงที่เลเยอร์ใดๆ ก็ต้องแก้ไขโค้ดทั้งหมดทุกครั้ง

3.2 สถาปัตยกรรมแบบสองเทียร์ (2-Tier Architecture)

ในสมัยก่อน ดิพลอยเมนต์ในระดับไฮเอนด์ใช้สถาปัตยกรรมแบบ 2 เทียร์ สถาปัตยกรรมนี้จะรวมบิซิเนสลอจิกเลเยอร์เข้ากับอีกหนึ่งในสองเลเยอร์ที่เหลือ ซึ่งทางเลือกที่เป็นไปได้คือรวมบิซิเนสลอจิกเลเยอร์เข้ากับพรีเซนเทชันเลเยอร์ และรวมบิซิเนสลอจิกเลเยอร์เข้ากับดาต้าเลเยอร์

■ การรวมบิซิเนสลอจิกเลเยอร์เข้ากับพรีเซนเทชันเลเยอร์

พรีเซนเทชันเลเยอร์ และบิซิเนสลอจิกเลเยอร์นั้น สามารถนำมารวมกันเป็นเทียร์เดียวได้ โดยแยกดาต้าแอ็กเซสเลเยอร์เป็นอีกหนึ่งเทียร์ต่างหาก ซึ่งเหมือนกับเป็นการสร้างกำแพงกั้นระหว่างบิซิเนสลอจิกเลเยอร์และดาต้าแอ็กเซสเลเยอร์ ดังรูปที่ 3-2

ถ้าหากเรามองว่าเทียร์แรกเป็นเทียร์ของไคลเอนต์ และเทียร์ที่สองเป็นเซิร์ฟเวอร์ สถาปัตยกรรมนี้จะทำให้ทางฝั่งไคลเอนต์มีขนาดใหญ่ (fat) ส่วนทางฝั่งเซิร์ฟเวอร์จะมีขนาดเล็ก (thin)

ในสถาปัตยกรรมแบบ 2 เทียร์ ไคลเอนต์แอปพลิเคชันจะติดต่อกับดาต้าเลเยอร์โดยผ่านทาง database bridge API เช่น Open Database Connectivity (ODBC) หรือ Java Database Connectivity (JDBC)

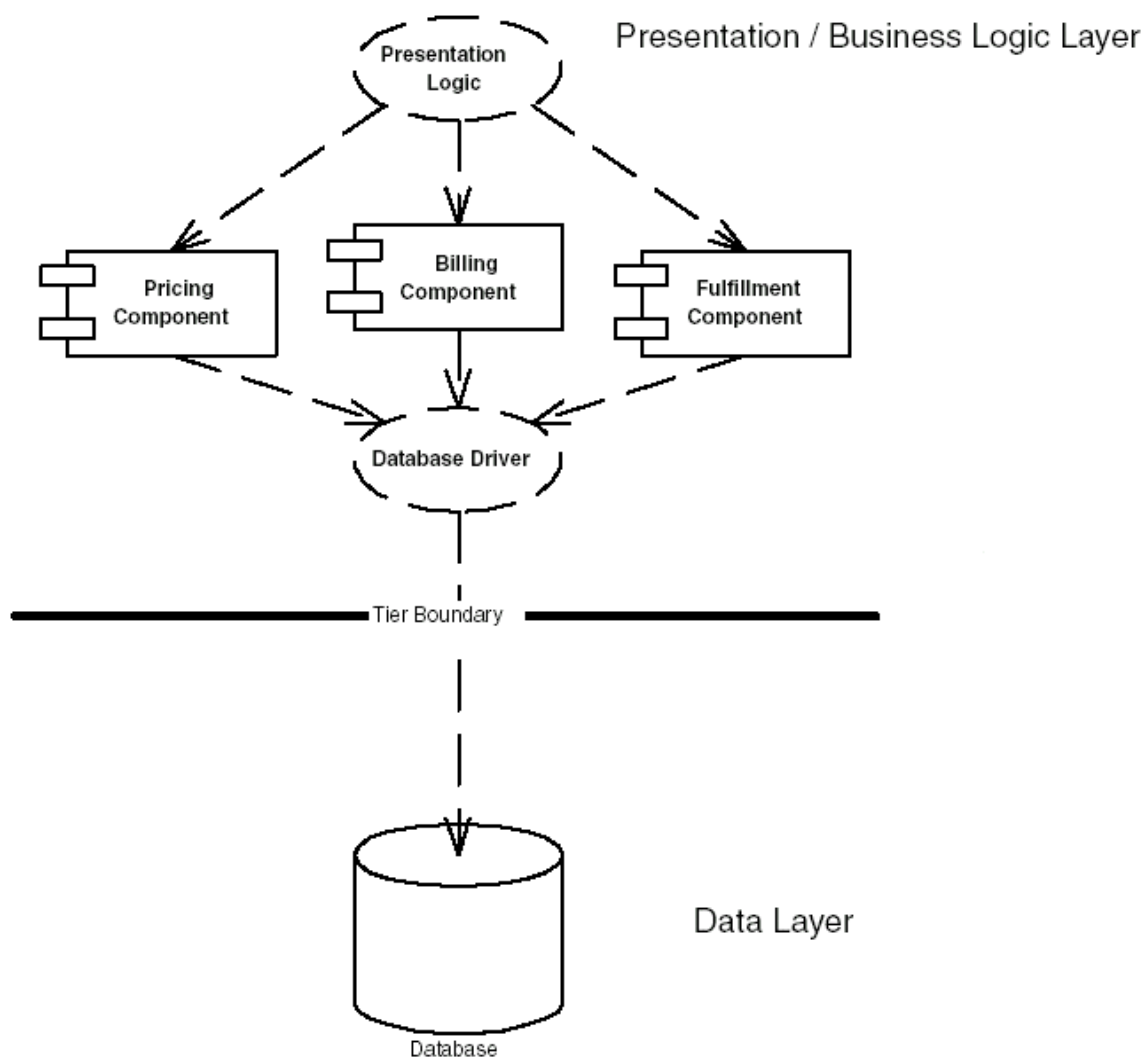
ลักษณะของสถาปัตยกรรมแบบสองเทียร์

- 1) มีค่าใช้จ่ายในการดีพลอยสูง

ไครเวอร์ของระบบฐานข้อมูลจะต้องถูกติดตั้งในไคลเอ็นต์ทุกเครื่องที่อยู่ในไคลเอ็นต์เพียง
ซึ่งอาจเป็นเครื่องคอมพิวเตอร์จำนวนหลายร้อย หรือหลายพันเครื่อง

2) มีค่าใช้จ่ายในการเปลี่ยนไครเวอร์ของระบบฐานข้อมูลสูง

การเปลี่ยนแปลงไครเวอร์ของระบบฐานข้อมูล จะต้องการการเปลี่ยนแปลงที่เครื่องไคลเอ็นต์ทุก
เครื่อง ซึ่งเป็นขั้นตอนที่สิ้นเปลืองมาก และเกือบจะเป็นไปไม่ได้ในความเป็นจริงสำหรับระบบงานที่มี
ขนาดใหญ่



รูปที่ 3-2 การรวมส่วนพีเรเซนต์เข้ากับบิซิเนสลอจิกในสถาปัตยกรรมแบบ 2 เทียร์

3) การเปลี่ยนแปลงโครงสร้างของฐานข้อมูลมีค่าใช้จ่ายที่สูง

เนื่องจากไคลเอ็นต์มีขนาดใหญ่ การเข้าถึงฐานข้อมูลเป็นการเข้าถึงโดยตรงโดยผ่านทาง JDBC ,
SQL/J หรือ ODBC ซึ่งหมายความว่า การเปลี่ยนแปลงโครงสร้างของฐานข้อมูลให้รองรับบิซิเนสลอ
จิกใหม่ ๆ จะต้องเปลี่ยนแปลงที่ไคลเอ็นต์ทุกเครื่องด้วย

4) การเปลี่ยนแปลงรูปแบบของฐานข้อมูลมีค่าใช้จ่ายที่สูง

ไคลเอนต์ที่มีขนาดใหญ่จะยึดติดอยู่กับ API ของระบบฐานข้อมูล เช่น API ของระบบฐานข้อมูลเชิงสัมพันธ์ (relational database) หรือ API ของระบบฐานข้อมูลแบบออบเจกต์ (object database) ถ้าหากเราตัดสินใจที่จะเปลี่ยนแปลงรูปแบบของฐานข้อมูล เช่น เปลี่ยนจากการใช้ฐานข้อมูลเชิงสัมพันธ์มาเป็นการใช้ฐานข้อมูลแบบออบเจกต์ นอกจากเราจะต้องทำการดีพลอยไคลเอนต์ทั้งหมดใหม่แล้ว ยังต้องมีการเปลี่ยนแปลงโค้ดภายในตัวไคลเอนต์ให้เข้ากับรูปแบบฐานข้อมูลที่เปลี่ยนแปลงไปอีกด้วย

5) การเปลี่ยนแปลงบิซิเนสลอจิกเลเยอร์มีค่าใช้จ่ายที่สูง

หากมีการเปลี่ยนแปลงที่เกิดขึ้นกับบิซิเนสลอจิกเลเยอร์ ไคลเอนต์จะต้องคอมไพล์ใหม่ และทำการดีพลอยใหม่ทั้งหมด

6) การเชื่อมต่อกับฐานข้อมูลมีค่าใช้จ่ายที่สูง

ไคลเอนต์แต่ละเครื่องจะติดต่อกับฐานข้อมูลโดยใช้ connection ของมันเอง ในขณะที่ connection มีจำนวนได้จำกัดและมีค่าใช้จ่ายในการสร้างสูง เมื่อไคลเอนต์ไม่ได้ใช้ฐานข้อมูล connection จะยังอยู่และ ไคลเอนต์อื่นไม่สามารถนำมันไปใช้ได้

7) ประสิทธิภาพของระบบเครือข่ายลดลง

ในแต่ละครั้งที่ไคลเอนต์ทำการติดต่อกับฐานข้อมูล จะมีการส่งข้อมูลจำนวนมากไปมา ระหว่างบิซิเนสลอจิกเลเยอร์ และดาต้าเลเยอร์ ซึ่งหากว่าตัวกลางระหว่างเลเยอร์ทั้งสองนี้เป็นระบบเครือข่าย อาจทำให้ข้อมูลในระบบเครือข่าวนั้นมากจนเกิดการติดขัด ซึ่งจะมีผลให้การติดต่อกับฐานข้อมูลนั้นทำได้ช้าลง

■ การรวมส่วนบิซิเนสลอจิกเลเยอร์บางส่วน เข้ากับดาต้าเลเยอร์

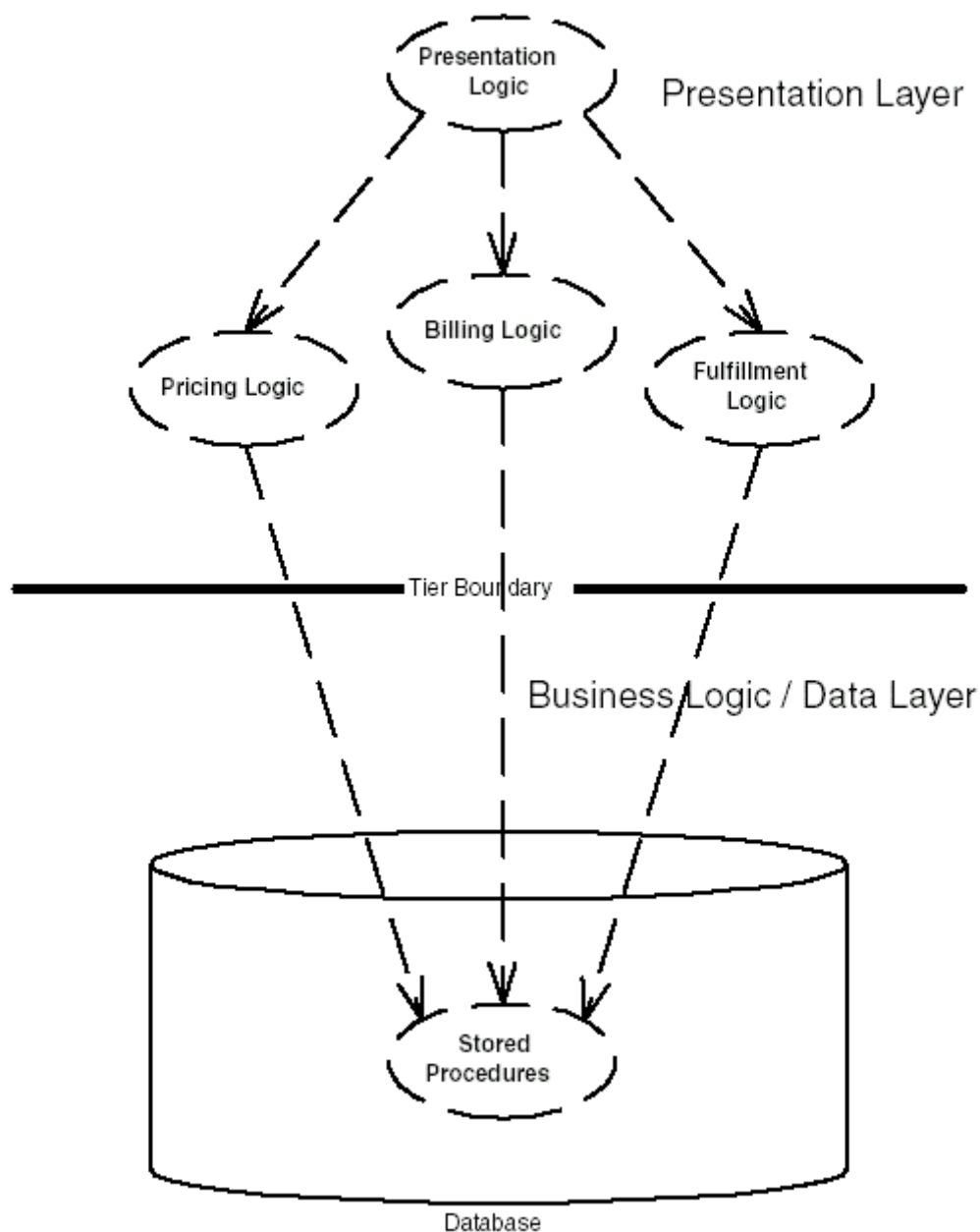
ในระยะหลัง ๆ มานี้ ดีพลอยเมนต์ต่าง ๆ จะนำบิซิเนสลอจิกเลเยอร์มารวมกับดาต้าเลเยอร์ และแยกไว้เป็นเทียร์หนึ่ง ซึ่งแสดงให้เห็นในรูปที่ 3-3 โดยหากมองว่าเทียร์ที่หนึ่งเป็นไคลเอนต์ ส่วนเทียร์ที่สองเป็นเซิร์ฟเวอร์ สถาปัตยกรรมแบบนี้จะทำให้ไคลเอนต์นั้นมีขนาดเล็ก ส่วนเซิร์ฟเวอร์นั้นจะมีขนาดใหญ่

ในสถาปัตยกรรมดังกล่าว จะนำบิซิเนสลอจิกบางส่วนเข้าไปไว้ในฐานข้อมูล ซึ่งโดยทั่วไปจะเป็นลอจิกที่เกี่ยวข้องกับข้อมูล ฐานข้อมูลจะอนุญาตให้เอ็กซิกิวต์ (execute) ลอจิกภายในฐานข้อมูลได้โดยการเขียนโมดูลที่เรียกว่า store procedure

การรวมลอจิกเข้าไปเป็น store procedure จะเป็นการเพิ่ม scalability และประสิทธิภาพในการทำงาน เนื่องจากว่าลอจิกบางส่วนถูกรวมเข้ากับฐานข้อมูล ดังนั้นการใช้งานเครือข่ายเพื่อติดต่อระหว่างลอจิกกับฐานข้อมูลจะลดลง แทนที่จะทำการ query ข้อมูลในฐานข้อมูลจำนวน n ครั้ง เราสามารถที่จะเรียก store procedure ซึ่งอยู่ภายในฐานข้อมูลให้ทำการ query n ครั้งแทน ซึ่งจะลดปริมาณการใช้เครือข่ายไปได้มากทีเดียว และยังทำให้การประมวลผลฐานข้อมูลเร็วขึ้นอีกด้วย

จะเห็นว่าการรวมลอจิกที่เกี่ยวข้องกับฐานข้อมูลเข้าไปไว้ในฐานข้อมูล ช่วยเพิ่มประสิทธิภาพโดยรวมของดีพลอยเมนต์ขึ้นอย่างมาก แต่บางปัญหาซึ่งเกิดกับสถาปัตยกรรมแบบ 2 เทียร์ก็ยังคงมีอยู่

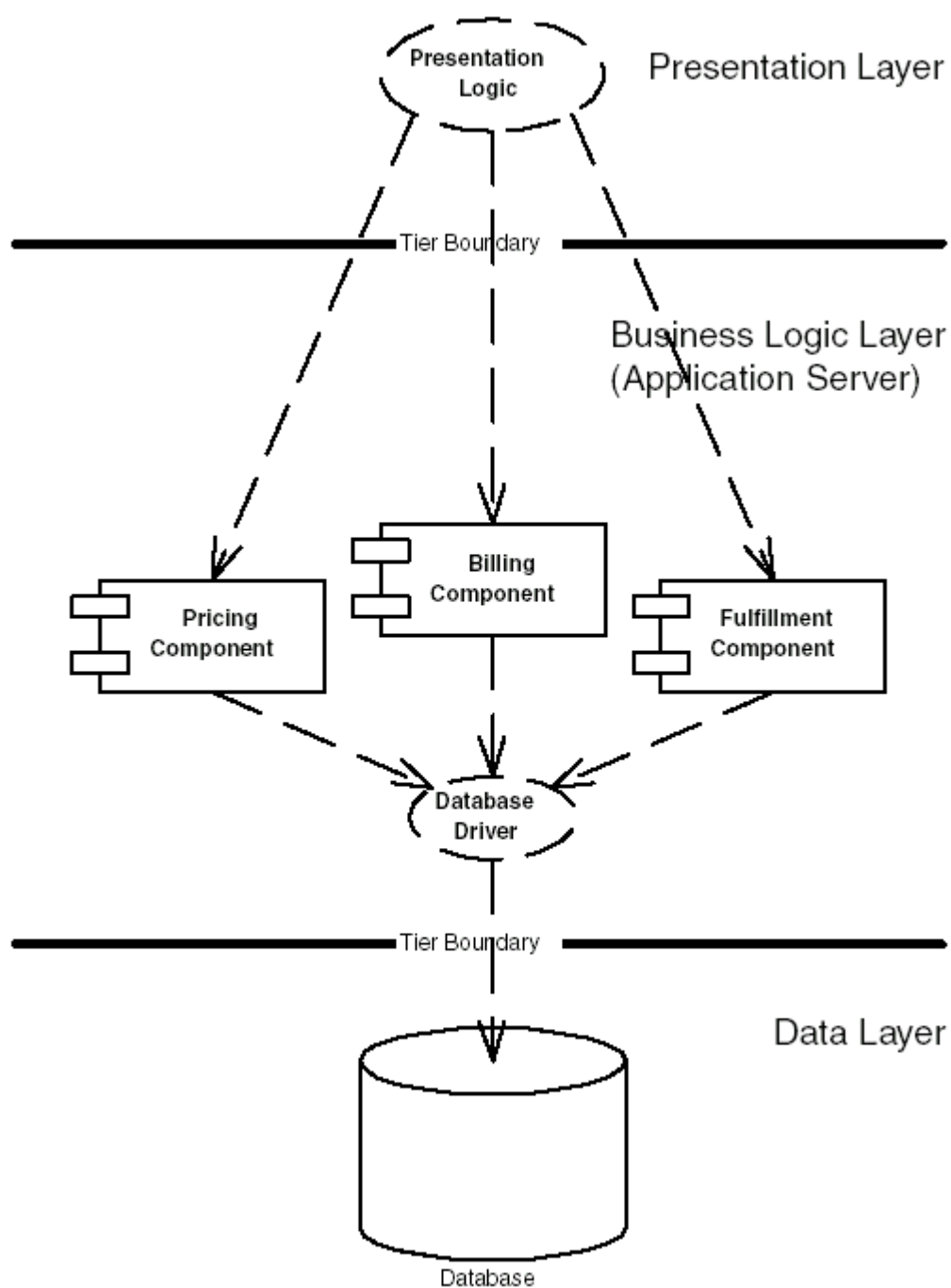
ในขณะการสร้าง store procedure นั้นเป็นวิธีการที่สามารถแก้ปัญหาได้หลายอย่าง แต่ตัวมันเองก็ทำให้เกิดปัญหาใหม่ ๆ ขึ้นมาเช่นกัน โดยภาษาที่ใช้ในการเขียน store procedure นั้น จะผูกอยู่กับฐานข้อมูลที่ใช้ ทำให้การเปลี่ยนแปลงระบบฐานข้อมูลที่ใช้อยู่นั้นทำได้ยาก แต่อย่างไรก็ตาม ภาษาจาวาได้ถูกนำมาใช้เขียน store procedure มากขึ้นเรื่อย ๆ ซึ่งจะช่วยเพิ่มความสามารถในการโยกย้าย (portability)



รูปที่ 3-3 การรวมบิซิเนสลอจิกเลเยอร์บางส่วน เข้ากับดาต้าเลเยอร์

3.3 สถาปัตยกรรมแบบสามเทียร์ (3-Tier Architecture)

สถาปัตยกรรมแบบการแยกเลเยอร์ทั้งสามออกจากกันไว้ในแต่ละเทียร์ ตัวอย่างที่ดีของสถาปัตยกรรมแบบสามเทียร์ได้แก่ web-base ดิสเพลย์เมนต์แบบ 3 เทียร์ ดังแสดงในรูปที่ 3-4 ซึ่งเทียร์ทั้งสามนั้น ถูกแบ่งออกดังนี้



รูปที่ 3-4 Web-base ดีพลอยเมนต์แบบ 3 เทียร์

- 프리젠테이션티어 (Presentation Tier)
 อาจประกอบไปด้วย Java Servlet , สคริปต์พวก Active Server Page หรือ Java Server Page
 และลอจิกที่เชื่อมการทำงานของแต่ละส่วนเข้าด้วยกัน
- บิซิเนสลอจิก티어 (Business Logic Tier)

แอปพลิเคชันเซิร์ฟเวอร์เป็นสิ่งจำเป็นในการให้สภาพแวดล้อมในการทำงานที่เหมาะสมกับคอมโพเนนต์ของบิซิเนสลอจิก แอปพลิเคชันเซิร์ฟเวอร์จะจัดหาเอ็นไวรอนเมนต์ให้กับบิซิเนสลอจิกคอมโพเนนต์ ทั้งยังทำหน้าที่จัดการคอมโพเนนต์ และให้บริการที่จำเป็นแก่คอมโพเนนต์เหล่านี้ ยกตัวอย่างเช่น แอปพลิเคชันเซิร์ฟเวอร์สามารถให้บริการติดต่อฐานข้อมูลกับ บิซิเนสคอมโพเนนต์ ซึ่งทำให้บิซิเนสคอมโพเนนต์สามารถเก็บข้อมูลแบบถาวร (persistent) ลงไป และดึงข้อมูลขึ้นมาจากค่าไคเียร์ได้ แอปพลิเคชันเซิร์ฟเวอร์ยังมีหน้าที่ในการจัดการคอมโพเนนต์โดยการสร้าง อินสแตนซ์ของคอมโพเนนต์เมื่อจำเป็น

- ค่าไคเียร์ (Data Tier)

อาจมีลอจิกเกี่ยวกับข้อมูลซึ่งถูกเก็บไว้ในรูปของ store procedure

ลักษณะของสถาปัตยกรรมแบบ N เทียร์

- 1) ค่าใช้จ่ายในการดีพลอยต่ำ

เนื่องจากไครเวอร์ของระบบฐานข้อมูลนั้นถูกติดตั้งอยู่ที่ฝั่งเซิร์ฟเวอร์ แทนที่จะเป็นเครื่องของไคลเอนด์ ซึ่งเป็นการสะดวกที่จะทำการติดตั้งซอฟต์แวร์บนฝั่งเซิร์ฟเวอร์ในสภาพแวดล้อมที่ควบคุมได้มากกว่าการติดตั้งบนเครื่องไคลเอนด์จำนวนมากมาย

- 2) ค่าใช้จ่ายในการเปลี่ยนแปลงระบบฐานข้อมูลต่ำ

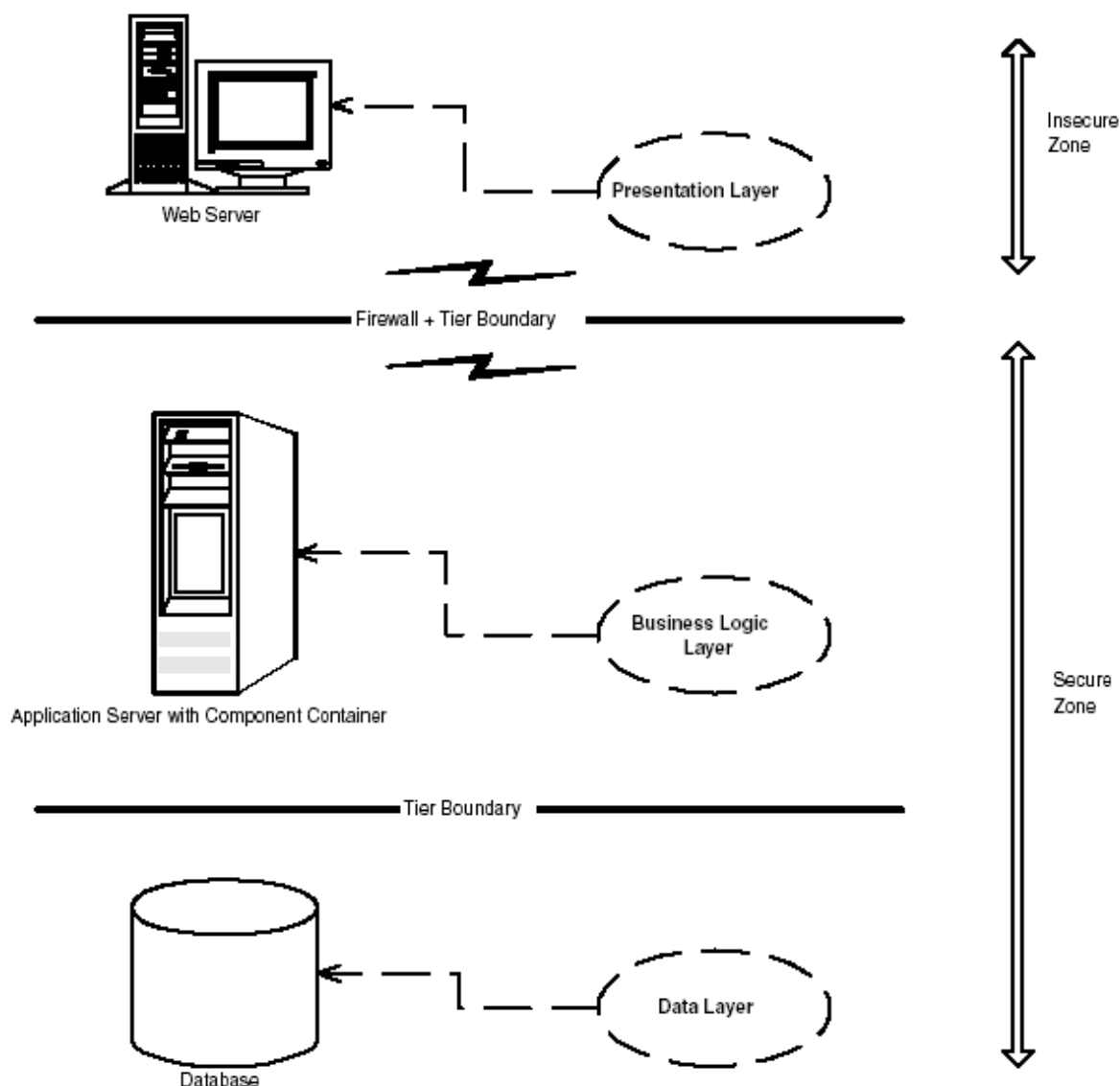
ไคลเอนด์จะทำการติดต่อกับฐานข้อมูลโดยผ่านทางเทียร์ซึ่งอยู่ตรงกลาง ทำให้เราสามารถเปลี่ยนแปลงโครงสร้างของฐานข้อมูล , ไครเวอร์ของฐานข้อมูล แม้กระทั่งฐานข้อมูลโดยไม่จำเป็นต้องดีพลอยไคลเอนด์ทั้งหมดใหม่อีกครั้ง

- 3) ค่าใช้จ่ายในการเปลี่ยนแปลงบิซิเนสลอจิกต่ำ

เพราะการเปลี่ยนแปลงในบิซิเนสลอจิกเลเยอร์ไม่จำเป็นที่จะต้องมีการดีพลอยไคลเอนด์เทียร์ใหม่

- 4) สามารถสร้างเขตความปลอดภัยให้กับส่วนใดส่วนหนึ่งของดีพลอยเมนต์ได้ด้วยไฟร์วอลล์

ใน web-base ดีพลอยเมนต์ เราอาจไม่ต้องการให้บุคคลภายนอกเข้ามาเห็นข้อมูลในบิซิเนสลอจิก เลเยอร์ แต่อนุญาตให้บุคคลภายนอกเข้าถึงพรีเซนเทชันเลเยอร์ได้ ซึ่งสามารถแก้ปัญหานี้ได้โดยการวางไฟร์วอลล์ไว้ระหว่างส่วนพรีเซนเทชัน และบิซิเนสลอจิกเทียร์ ดังแสดงในรูปที่ 3-5

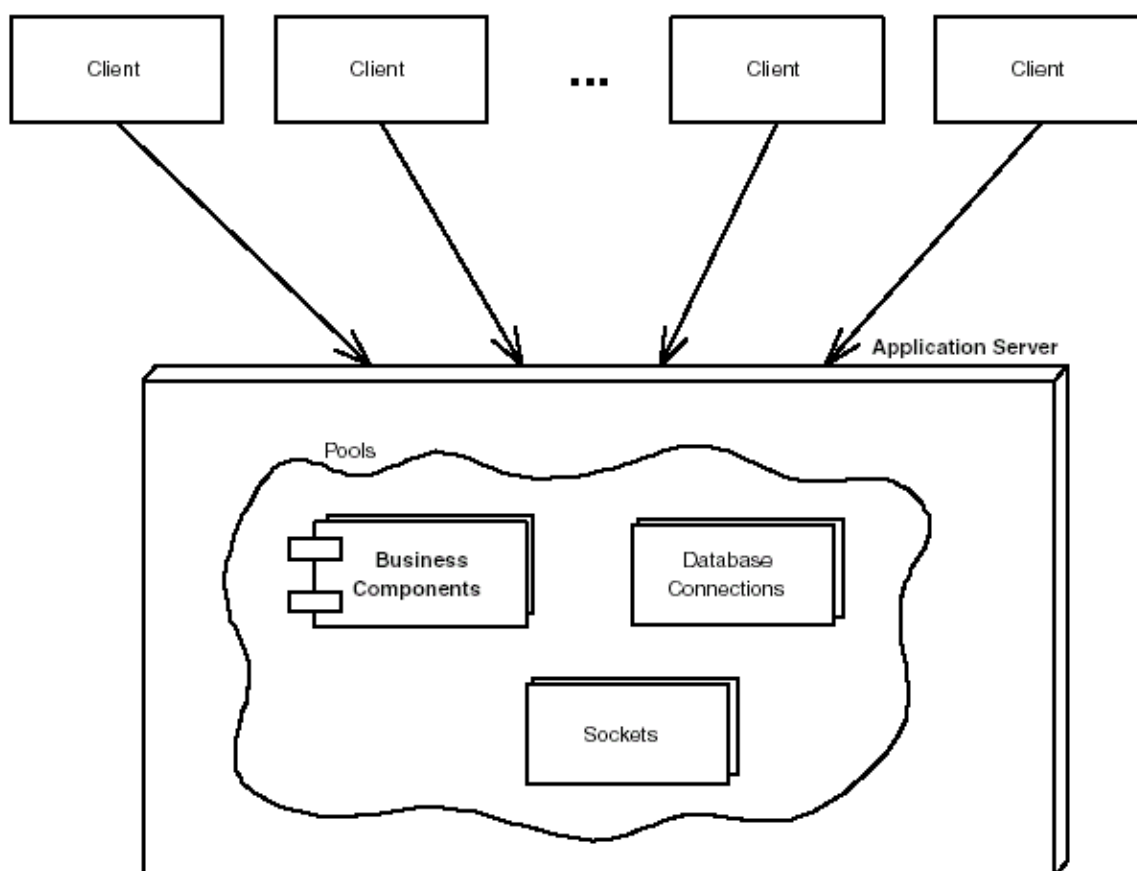


รูปที่ 3-5 การรักษาความปลอดภัยในสถาปัตยกรรมแบบ 3 เทียร์

5) ทรัพยากรสามารถถูกพูล (pool) และนำกลับมาใช้ใหม่ได้อย่างมีประสิทธิภาพ

ในสถาปัตยกรรมแบบสามเทียร์ การเชื่อมต่อไปยังทรัพยากรภายนอกสามารถถูกจัดการได้อย่างมีประสิทธิภาพ การพูลทรัพยากร (resource pooling) นั้นใช้หลักความเป็นจริงที่ว่า ไคลเอนต์มักจะทำงานอื่นไปพร้อม ๆ กับการใช้ทรัพยากร เช่น การสร้างส่วนติดต่อกับผู้ใช้ แทนที่บีซิเนสคอมโพเนนต์จะทำการเชื่อมต่อ และปล่อยการติดต่อกับทรัพยากร (เช่น ฐานข้อมูล) ทรัพยากรนั้นสามารถที่จะถูกลูและนำไปให้บริการแก่ไคลเอนต์หลาย ๆ เครื่องได้ ทำให้จำนวนของการเชื่อมต่อฐานข้อมูล (database connection) นั้น จะมีจำนวนน้อยกว่าจำนวนของคอมโพเนนต์ทั้งหมดมาก

เนื่องจากการติดต่อกับฐานข้อมูลนั้นเป็นวิธีการที่แพง (expensive) วิธีการนี้จะช่วยเพิ่ม scalability ของดีพลอยเมนต์ นอกจากนี้แล้วการเชื่อมต่อไปยังทรัพยากรนั้นไม่จำเป็นที่จะต้องถูกสร้างขึ้นใหม่เรื่อย ๆ ซึ่งจะช่วยเพิ่มประสิทธิภาพการทำงานของแอปพลิเคชัน วิธีการพูลทรัพยากรนี้สามารถใช้กับทรัพยากรประเภทอื่น ๆ นอกเหนือจากฐานข้อมูลได้อีกด้วย เช่น การเชื่อมต่อซ็อกเก็ต (socket connection) หรือเธรด (thread) จริง ๆ แล้วในสถาปัตยกรรมแบบสามทีเยร์นั้น บิซิเนสคอมโพเนนต์เองสามารถที่จะพูลและให้บริการแกไคลเอนต์หลาย ๆ เครื่องได้ การพูลคอมโพเนนต์นั้นหมายความว่าเราไม่จำเป็นที่จะต้องมียอมโพเนนต์ส่วนตัวสำหรับไคลเอนต์แต่ละเครื่อง การพูลทรัพยากรนั้นถูกแสดงไว้ในรูปที่ 3-6



รูปที่ 3-6 การพูลทรัพยากร (Resource Pooling) ในสถาปัตยกรรมแบบสามทีเยร์

6) ประสิทธิภาพถูกแบ่งเป็นส่วน ๆ

หากทีเยร์ใดทีเยร์หนึ่งเกิดการโอเวอร์โหลด (overload) ขึ้น ทีเยร์อื่น ๆ จะยังคงทำงานต่อไปได้อย่างปกติ เช่นในกรณีของเว็บดีพลอยเมนต์ ผู้ใช้สามารถที่จะเข้ามายังหน้าแรกของเว็บไซต์มาก ถึงแม้ว่าที่แอปพลิเคชันเซิร์ฟเวอร์จะเกิดการโอเวอร์โหลดก็ตาม

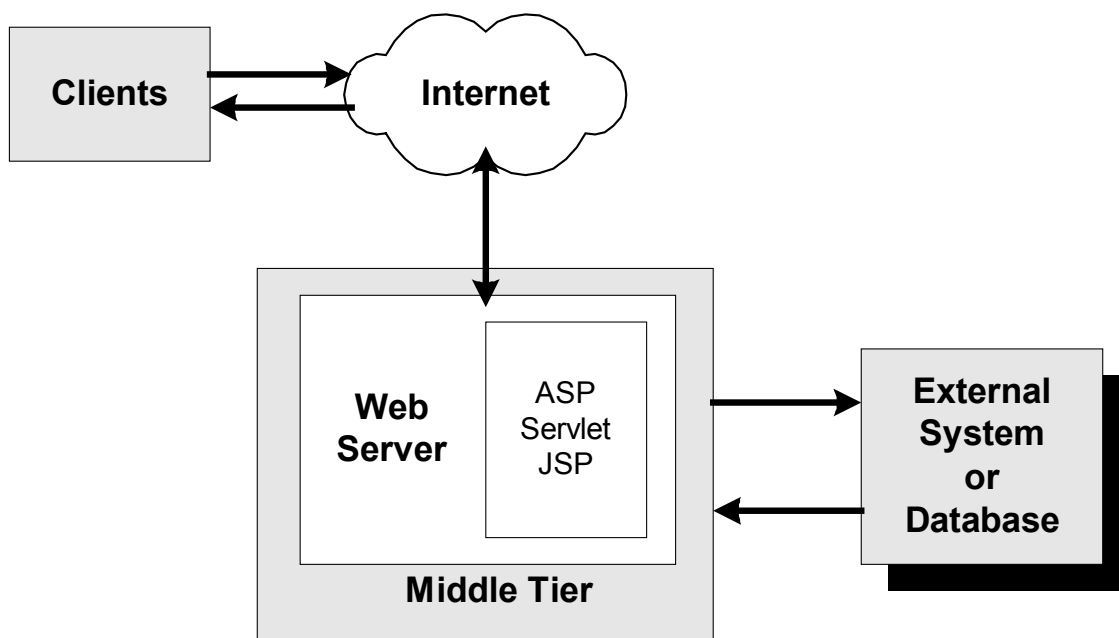
7) ความผิดพลาดในการทำงานถูกแบ่งเป็นส่วน ๆ

หากเกิดความผิดพลาดในการทำงานอย่างรุนแรงขึ้นที่-tierใด ๆ ความผิดพลาดก็จะถูกจำกัดวงอยู่เพียงใน-tierนั้นโดยไม่มีผลกระทบต่อ-tierอื่น ๆ ยกตัวอย่างเช่น หากแอปพลิเคชันเซิร์ฟเวอร์เกิดความผิดพลาดในการทำงาน หรือแครช (crash) เว็บเซิร์ฟเวอร์สามารถทำการรายงานความผิดพลาดให้กับผู้ใช้ทราบได้

8) มีค่าใช้จ่ายในการบำรุงรักษาสูง

เนื่องจากเราต้องทำการดีพลอยระบบถึงสาม-tierขึ้นไปในทางกายภาพ ทำให้ค่าใช้จ่ายในการติดตั้งซอฟต์แวร์, การอัปเดตซอฟต์แวร์, การดีพลอยใหม่ และค่าใช้จ่ายในการบำรุงรักษาระบบสูงขึ้น

web-base ดีพลอยเมนต์แบบ 3 tier หากมองแบบไคลเอนต์/เซิร์ฟเวอร์ดีพลอยเมนต์แล้วจะกลายเป็น 4 tier เนื่องจากการแบ่งเป็น-tierเป็นการแบ่งทางกายภาพที่ขึ้นอยู่กับว่าแบ่งโดยสิ่งใด ใน web-base ดีพลอยเมนต์จะแบ่งโดยใช้หน้าที่ที่แต่ละส่วนทำในระบบเว็บคือ ส่วนแสดงผล ส่วนประมวลผลบิซิเนสลอจิก และส่วนเก็บข้อมูล แต่แบบไคลเอนต์/เซิร์ฟเวอร์ดีพลอยเมนต์จะแบ่งโดยใช้งานที่เครื่องทำเป็นหลัก แบ่งเป็น ไคลเอนต์-tier, มิดเดิล-tier และดาต้า-tier โดยมิดเดิล-tierอาจจะเป็นเว็บเซิร์ฟเวอร์อย่างเดียว หรือมีแอปพลิเคชันเซิร์ฟเวอร์ด้วยก็ได้ โดยหากมีเว็บเซิร์ฟเวอร์อย่างเดียวก็จะเป็นระบบไคลเอนต์/เซิร์ฟเวอร์ดีพลอยเมนต์แบบ 3 tier ดังรูปที่ 3-7 หากมีแอปพลิเคชันเซิร์ฟเวอร์ด้วยจะเป็นระบบไคลเอนต์/เซิร์ฟเวอร์ดีพลอยเมนต์แบบ 4 tier ดังรูปที่ 3-8

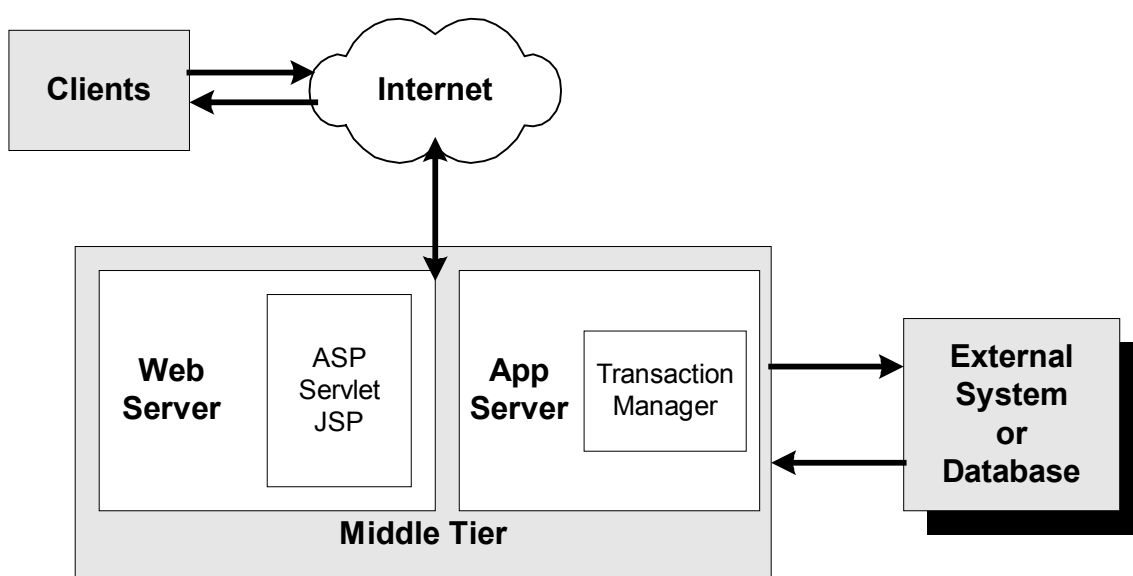


รูปที่ 3-7 ระบบไคลเอนต์/เซิร์ฟเวอร์ดีพลอยเมนต์แบบ 3 tier

3.4 สถาปัตยกรรมแบบ N เทียร์ (N-Tier Architecture)

สถาปัตยกรรมแบบเอ็นเทียร์เป็นการเพิ่มเทียร์หนึ่งหรือมากกว่าลงไปในโมเดลแบบ 2 เทียร์ ในดีพลอยเมนต์แบบเอ็นเทียร์ ฟรีเซนต์เทชันเลเยอร์, บิซิเนสลอจิกเลเยอร์ และดาต้าเลเยอร์ จะถูกแบ่งไว้ในแต่ละเทียร์ ในสถาปัตยกรรมแบบ 4 เทียร์หรือมากกว่า จะเป็นการแบ่งเลเยอร์เหล่านี้ลงไปอีก เพื่อให้แต่ละส่วนของระบบนั้นสามารถเพิ่มขยายได้โดยอิสระจากกัน

ตัวอย่างที่ดีของสถาปัตยกรรมแบบ N เทียร์ได้แก่ web-base ดีพลอยเมนต์แบบ 3 เทียร์ ดังแสดงในรูปที่ 3-4 ซึ่งเทียร์ทั้งสามนั้น ถูกแบ่งออกดังนี้



รูปที่ 3-8 ระบบไคลเอนต์/เซิร์ฟเวอร์ดีพลอยเมนต์แบบ 4 เทียร์